

Assignment 3: Inference Systems

CAS CS 320: Concepts of Programming Languages

due Thursday, September 24 by 8:00PM

For the OCaml problems, you'll put your solutions in the file `lib/assign3.ml`. See the file `test/test_assign3.ml` for example behavior of each function. You'll submit your solutions via Gradescope, where your code will be checked against a collection of hidden tests. You must pass at least a third of the gradescope unit tests on each problem to for the assignment to be considered attempted. Your code must also follow the guidelines of the course for it to be considered attempted (i.e., it must use the course standard library, it may not use any references, arrays, mutable fields, for/while loops).

For the written problems, you'll submit your solutions via Gradescope as a single PDF file. They must be hand-written and exceptionally neat, and the final answer to each problem must be clear, e.g., surrounded by a box. You must cite your sources per the instructions in the course manual.

You're required to submit solutions to every problem in this assignment to receive full credit, but we will only provide detailed feedback on **bolded** problems. Since the assignment is graded on participation, we ask that you *try at least the bolded problem with as few external aids as possible*. You should think of these problems as simulated quizzes or exams. *You won't be penalized for mistakes*, so it's in your best interest to do these problems on your own; the detailed feedback we provide will generally be more useful (for you and for us) if you submit your own work.

OCaml Exercises

These problems are from the OCaml Exercises linked on the course webpage.

2.17 Nub

2.29 Split by Whitespace

Written Problems

1. Write down an English sentence which expresses the same thing as the following typing judgment. (Note: This operators is not in the OCaml standard library)

$$\{f : \tau_1 \rightarrow \tau_2, g : \tau_2 \rightarrow \tau_3\} \vdash f \gg g : \tau_1 \rightarrow \tau_3$$

2. Write down an English sentence which expresses the same thing as the following typing rule.

$$\frac{\Gamma \vdash e_1 : \text{string} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1.[e_2] : \text{char}} \text{STRINDEX}$$

- (3) Suppose we wanted to introduce a `xor` as an operator in OCaml. The syntax of this operator is given by:

$$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle \text{ xor } \langle \text{expr} \rangle$$

and the typing is given by a single inference rule:

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \text{bool}}{\Gamma \vdash e_1 \text{ xor } e_2 : \text{bool}} \text{XOR}$$

There are two semantic rules for this operator, depending on if the first argument evaluates to $\perp$ or $\top$. Here is the $\perp$ case.

$$\frac{e_1 \Downarrow \perp \quad e_2 \Downarrow b}{e_1 \text{ xor } e_2 \Downarrow \neg b} \text{XORFALSEE}$$

where $\neg b$ is the *negation* of b . Write down the second semantic rule for `xor`.

4. Using only anonymous functions, applications, and tuples, determine an expression of type `('a * 'b -> 'c) -> 'a -> 'b -> 'c`.
5. Using only anonymous function and applications, determine an expression of type `('a -> 'a -> 'a) -> 'a -> 'a`. Note that this type is *not* the same as the type `('a -> 'a -> 'b) -> 'a -> 'b`.

6. What type can `g` have so that the following expression is well-typed? Can `g` be have a polymorphic type? Also, what is the type of the following expression? Justify your answer.

```
let rec f x = g x + g (f x) in f
```

7. Suppose that

- `h` has type `'a -> 'a list -> 'a` and
- `k` has type `'a -> ('b -> 'a) list`.

Determine the type of the following expression. Justify your answer.

```
fun x y -> h x (k y)
```

- (8) Determine the type τ and the minimal context Γ such that the following typing judgment holds in OCaml. (Hint. Come up with dummy values for the unknown variables in the following expression so that you can type it into UTop and it will type-check.)

$$\Gamma \vdash \text{let rec } f \ x = f \ (x + g \ (f \ y)) \text{ in } h \ (g \ (x = y)) = f \ x : \tau$$